

UTMStack EDR — Completion Plan and Team Assignments

Date: 2026-09-21 **Product owner:** Rick **Engineers:** Yadian · Alex · Andres · Jose

"EDR" here means the UTMStack endpoint detection and response module: the piece that watches an endpoint for malware and hostile behaviour, stops it, and reports what it did.

This document says three things: what already works, what is still missing before we can call the feature finished, and exactly who builds each missing piece.

1. Where the work stands today

1.1 The endpoint module (Windows) — built and proven on a test machine

Roughly 11,700 lines of Go (3,650 of them tests) plus 178 lines of native C++ for the one piece that cannot be written in Go. Every package's tests pass, the module cross-builds and passes static checks for both Windows processor families (Intel/AMD 64-bit and ARM 64-bit), and the branding audit is clean.

Capability	What it does	State
Own service	<code>utmstack_edr.exe</code> runs as its own Windows system service; the agent only starts, stops and configures it	Working, proven on the test machine
Scanning engine	The open-source scanning engine runs as a supervised child process; the module writes its configuration and tunes it to the host's memory and core count	Working, proven
File arrival watch	Reads the file system change journal for the whole volume, scans new and changed files, moves malicious ones to quarantine (never deletes), supports restore	Working, proven
Process launch watch	Sees every process start, scans its image, and kills the whole process tree when the verdict is malicious. Optional freeze-before-scan exists but ships off because it once froze the test machine	Working, proven
Script blocking	A small native library plugs into the Windows script-scanning interface and forwards script text to the module before it runs; a real malicious script was blocked end to end	Working, proven
Behavioural telemetry	Forwards process creation and PowerShell script-block records to the platform	Working, proven
Ransomware guard	Decoy files plus recovery-tampering command rules plus a file-activity feed drive a per-process score; crossing a threshold freezes, kills and quarantines the culprit	Working, proven. On by default in freeze mode
Network blocking	Pulls indicator lists from the server mirror and blocks matching addresses, address ranges and domains using the built-in Windows filtering layer, with self-lockout protection	Working, proven. On and enforcing by default
Configuration	One file, <code>edr.json</code> . A local command-line tool edits it. Changes to allowlists and response mode apply live; structural changes need a restart	Working
False-positive tuning	One combined allowlist for paths, processes, commands and networks, plus a switch per sensor so one noisy detector can be turned off without disabling the module	Working
Event path	The module appends events to a durable spool file; the agent drains it into the existing log channel, so there is one outbound connection and at-least-once delivery	Working, proven

1.2 The master-server side — built, not deployed

A new container called `edr` (about 1,600 lines of Go) mirrors two content channels for the whole fleet from one upstream fetch: the scanning engine's signature databases and the threat-intelligence indicator lists. Agent-manager serves the shared volume read-only on the existing dependencies port for deployments with a trusted certificate; the container serves the same tree over plain HTTP on port 9002 for self-signed deployments, where the signature updater cannot validate certificates. The installer and the build pipeline are wired.

It has never run on a real customer deployment.

1.3 What is missing

1. **Nothing in the UTMStack console shows or controls the EDR.** Today an administrator would have to open a remote shell to each endpoint and type commands. There is no page, no policy, no fleet view, no remote quarantine.
2. **The platform does not understand the events.** The endpoints already send them, but there is no parser, no correlation rule and no playbook, so they land in the log explorer as unstructured records and never become alerts.
3. **Linux is stubs only.** Every sensor compiles on Linux and does nothing.
4. **The binaries are not delivered.** The build pipeline does not publish `utmstack_edr` and there is no code anywhere that downloads the scanning engine — it was installed by hand on the test machine.
5. **Nothing is signed.** Production Windows deployment needs an Authenticode signature on both the module and the native script-scanning library.
6. **Two real endpoint gaps for a complete product:** there is no scheduled full-disk scan, and there is no host isolation (cutting a compromised endpoint off the network while leaving the path back to UTMStack open).

1.4 Code location

`utmstack/OpenEDR` (private) is the working home for this project. It is a full working copy of the UTMStack v12 tree, because nearly every remaining task touches `backend/`, `frontend/`, `agent-manager/`, `installer/` or `definitions/` as well as `agent/edr/`.

Branch	What it is
<code>main</code>	Default. Everyone works here. <code>release/v12.0.0</code> with both EDR branches merged
<code>edr-phase1</code>	Endpoint module: all of <code>agent/edr/</code> plus the thin agent hooks
<code>feature/edr-server-container</code>	Master-server container plus its agent-manager, installer and pipeline wiring
<code>release/v12.0.0</code>	Mirror of the upstream base, so going upstream later stays a normal pull request
<code>snapshot-archive</code>	The old flat safety snapshot, superseded

Designs, plans and the original requirement documents are in `docs/edr/` on `main`. `utmstack/UTMStack` (public) remains the eventual upstream; drop `docs/edr/` from any pull request that goes there, since it is internal planning.

2. What "done" means

The feature is finished when all of the following are true.

1. An administrator can see, from the UTMStack console, which endpoints have the EDR, whether it is healthy, how fresh its signatures are, and what it has caught.

2. **Every setting that exists on an endpoint is reachable from the console** — not a curated subset — and can be applied to the whole fleet, to a group, or to selected endpoints, with the more specific policy winning and the resolved result visible before saving.
3. **A false positive is fixed once, for the whole company.** An administrator marks a detection as benign, chooses how widely that applies, and the file is restored everywhere it was quarantined. Nobody ever has to disable the module to get work done.
4. An administrator can act on an endpoint from the console without a shell: run a scan, restore or permanently remove a quarantined file, kill a process, isolate the host.
5. There is a **dedicated EDR dashboard** with per-workstation status and health, of the kind a customer would compare against a purpose-built endpoint product's console.
6. Detections become alerts, with correlation rules and automated response playbooks.
7. Everything above works the same on Windows and on Linux.
8. The signature and threat-intelligence mirrors run on the customer's own server, and the endpoint binaries are published, signed and installed by the normal agent update path.
9. We have measured what the product actually stops, on both operating systems, and written down honestly what it does not.

3. How the work is split

Engineer	Owns	Why
Yadian	The control path between the server and the agent, and the Linux port of the endpoint module	He knows the agent manager and the agent-to-server conversation better than anyone, and the Linux sensors are agent-internal work
Alex	The backend module and its interface, the master-server container in production, the build and delivery pipeline, and the assistant integration	Large volume of self-contained backend work that does not require deep agent knowledge
Andres	Everything the administrator sees	Front-end owner; picks up light interface glue on the backend where the screens need shaping
Jose	Detection content and proving every capability works on both operating systems	He is the person for parsers, correlation and cross-platform behaviour verification

Nobody works alone on a boundary: Yadian and Alex agree the message shapes before either starts; Andres and Alex agree the interface shape before the screens start; Jose reviews every capability before it is called complete.

4. Phases

Every task has a hard finish date. These are commitments, not estimates.

Phase	Goal	Finishes
Phase 0 — Land what exists	Branches merged and building, binaries published, the engine installs itself, events become alerts	Fri 2 Oct 2026
Phase 1 — Console control, Windows	An administrator can see and drive the EDR from UTMStack on Windows endpoints	Fri 23 Oct 2026
Phase 2 — Linux parity	Every capability works on Linux and is visible in the same screens	Fri 20 Nov 2026
Phase 3 — Production hardening	Signed binaries, scale proven, known defects closed, effectiveness campaign, documentation	Fri 4 Dec 2026
Phase 4 — Next capabilities	Deferred work that still has a committed date: second-round ransomware sensors, application control, the defect backlog	Fri 15 Jan 2027

Phases 1 and 2 overlap: Yadian moves to Linux as soon as the control path is handed to Alex and Andres for wiring. Section 12 is the full dated task list, and every task is a GitHub issue in `utmstack/0penEDR` on the org board.

5. Phase 0 — Land what exists (2 weeks)

Yadian

Y0.1 — Review and merge the endpoint branch. Read `edr-phase1` (two commits, about 135 files) against `release/v12.0.0`, focusing on the four files that touch agent packages: `agent/cmd/{enable_edr,disable_edr,edr_status}.go`, `agent/agent/edr_relay.go`, `agent/dependency/edr.go` and the one line added to `agent/serv/service.go`. Confirm the module stays self-contained and that a failure inside it cannot take the agent down. *Done when:* the branch is pushed, a pull request into `release/v12.0.0` is approved, and the agent still starts cleanly with the EDR absent, present-but-disabled, and enabled.

Y0.2 — Review and merge the server container branch. `feature/edr-server-container` touches agent-manager, the installer and the build pipeline. Check the new static route on the dependencies port cannot expose anything beyond the mirror volume and that directory listings stay refused. *Done when:* merged, and a fresh install produces a stack that includes the `edr` service.

Alex

A0.1 — Publish the endpoint binaries through the build pipeline. The build currently produces nothing for the EDR. Add build steps that produce `utmstack_edr_windows_amd64.exe` and `utmstack_edr_windows_arm64.exe` and stage them into the agent-manager dependencies image the same way the collector is staged (`agent-manager/Dockerfile` copies `./dependencies/agent/`). *Done when:* a built agent-manager image serves both files at `https://<server>:9001/private/dependencies/agent/`, and an agent with `edr` in its dependency list downloads and installs the module by itself.

A0.2 — Package and host the scanning engine. No code anywhere downloads the engine; it was unpacked by hand on the test machine. Produce a per-platform engine package (the daemon, its support libraries and a seed database set), host it at the same dependencies endpoint, and write the endpoint-side code that downloads and unpacks it into the module's `engine/` directory when the module is enabled and the directory is missing or incomplete. Reuse `shared/http.DownloadFile` and `shared/archive.Unzip`. *Done when:* a clean Windows machine with only the agent installed ends up with a working engine after `enable-edr`, with no manual steps.

A0.3 — Deploy the server container on a real deployment and watch it for a week. Bring up the `edr` service on a staging server, confirm the signature mirror fills, confirm the threat-intelligence lists download with the platform's stored credentials, and confirm agents pull from it on both transports (trusted certificate on port 9001, self-signed on port 9002). Settle the one open question in the design: whether the upstream feed path expects `level1` or `1`. *Done when:* an endpoint's signature database updates from the customer's own server with no internet access from the endpoint, and the mirror's freshness document is being written.

Jose

J0.1 — Write the parser for EDR events. Create `definitions/filters/utmstack/utmstack_edr.yaml`. The events arrive as one JSON object per record with data type `utmstack_edr`. Fields: `timestamp`, `host_id`, `os`, `product`, `source` (one of `file_watcher`, `process_watcher`, `amsi`, `edr_engine`, `behavioral`, `network_watcher`), `action` (`detected`, `quarantined`, `killed`, `blocked`, `allowed`, `scan_complete`, `health`, `ransomware_suspected`, `ransomware_contained`), `object_path`, `file_hash`, `verdict`, `signature`, `remote_ip`, `remote_port`, `domain`, `direction`, `indicator`, `engine`, `severity`, and a nested `process` object with `pid`, `ppid`, `image`, `command_line`, `session`, `user`. *Done when:* every one of the six sources produces correctly typed, searchable fields in the log explorer, with no unparsed remainder.

J0.2 — Write the first correlation rules. Create `definitions/rules/edr/`. At minimum: malware detected and quarantined; malware detected but not contained; malicious process tree terminated; script blocked before execution; ransomware behaviour suspected; ransomware contained; connection blocked to a known-bad address; repeated blocked connections from one host; scanning engine unhealthy; signature database stale; a sensor switched off on an endpoint that previously had it on (tamper signal); a quarantined file restored (audit trail). *Done when:* each rule fires on a real event produced by the test machine, and none of them fire during an hour of ordinary desktop use.

J0.3 — Alert severity and scoring. Map each rule to a severity and make sure the scoring module treats a contained ransomware incident as higher-impact than a single quarantined file. *Done when:* Rick agrees the severities read correctly on the alerts page.

Andres

N0.1 — Detections view. Before any control surface exists, administrators need to see what the EDR found. Add an EDR section to the alerts and log explorer experience that presents the parsed fields well: file path, verdict, signature name, the process tree, and for network blocks the address, domain and matched indicator. *Done when:* an EDR alert opens into a readable detail panel without raw JSON.

6. Phase 1 — Console control on Windows (6–8 weeks)

This is the heart of the request: make the EDR a managed feature of UTMStack rather than a thing that runs on its own.

6.1 The control path — Yadian

Today the only way the server can make an endpoint do something is to send a shell command string over the agent stream and read back its text output (`agent-manager/protos/agent.proto` , `UtmCommand` and `CommandResult` ; handled in `agent/agent/incident_response.go`). That is fine for an interactive console and wrong for a product feature: no typed result, no way to tell a failure from empty output, and the audit trail is a shell line.

Recommended approach: add typed messages to the same stream. One connection, one authentication path, no shell.

Y1.1 — Define the message contract. In `agent-manager/protos/agent.proto` add to the `BidirectionalStream` oneof:

- `EdrCommand { agent_id, cmd_id, action, payload, executed_by, reason }` where `action` is one of `enable` , `disable` , `status` , `policy_set` , `quarantine_list` , `quarantine_restore` , `quarantine_purge` , `scan_path` , `full_scan` , `kill_process` , `allow_add` , `allow_remove` , `blocklist_refresh` , `isolate` , `release_isolation` , and `payload` is a JSON argument object.
- `EdrResult { agent_id, cmd_id, ok, error, payload, executed_at }` where `payload` is a JSON result object.
- `EdrStatusReport { agent_id, status_json, policy_version, reported_at }` sent by the agent without being asked.

Agree the shape with Alex before generating code, because his interface mirrors it. *Done when:* generated code compiles in agent-manager, the agent and the backend client, and an old agent that does not understand the new messages still works.

Y1.2 — Machine-readable output on the endpoint command-line tool. `agent/edr/manage.go` already implements `config show|get|set` , `allow ...` , `quarantine list|restore|purge` and `main.go` implements `scan` , `status` , `restore` . Add a `--json` flag so every verb can emit a structured result instead of human text. This reuses a surface that is already tested on the test machine rather than inventing a new channel between the agent and the module. *Done when:* every verb returns valid JSON with a stable field set, and the existing human output is unchanged when the flag is absent.

Y1.3 — Agent-side handler. New file `agent/agent/edr_control.go` . It receives `EdrCommand` , maps the action onto either service control (`shared/svc`), a configuration write (`agent/edr/config`) or an invocation of the module's command-line tool with `--json` , and returns `EdrResult` . It must never block the stream: long actions such as a full scan return "accepted" immediately and report completion through an event. *Done when:* every action in Y1.1 round-trips from a test client, including the failure cases (module not installed, service stopped, file already restored).

Y1.4 — Status reporting upstream. The module already writes `status.json` with engine health, signature version and source, sensor states, ransomware state, network blocking state and engine tuning. Have the

agent send `EdrStatusReport` every five minutes and immediately after any change, and have agent-manager store the latest report per agent. *Done when:* stopping the scanning engine on the test machine turns that endpoint unhealthy in the server's stored state within five minutes.

Y1.5 — Policy distribution. A policy is the centrally managed subset of `edr.json`: enabled, sensor switches, allowlists, ransomware mode and thresholds, network blocking settings, quarantine retention, scan concurrency, signature source, engine tier override. The server sends `policy_set` with the policy document and a version string; the agent merges it into `edr.json`, saves, and lets the module's live reload pick up what it can. Settings that need a restart must cause the agent to restart the module. The agent reports the applied policy version in every status report so the server can show drift.

Two rules that must not be broken: the endpoint must keep working with its last policy when the server is unreachable, and a local administrator's emergency change must not be silently overwritten without it being visible as drift in the console. *Done when:* changing a policy centrally changes behaviour on the test machine, with allowlist changes taking effect within seconds and structural changes surviving a restart.

Y1.6 — Host isolation. A real EDR can cut an endpoint off. Build it on the same Windows filtering layer the network blocking already uses: block all traffic except the UTMStack server, the agent's ports, and the loopback interface. It must fail open — if the module dies, isolation lifts — and it must be reversible from the console and from the local command-line tool. *Done when:* an isolated test machine cannot reach the internet or the local network, still reports to UTMStack, and comes back fully when released.

Y1.7 — Scheduled full scan. There is no scheduled scan today. Add one driven by policy: a schedule, a target set of paths, and a throttle so it does not swamp the machine. Emit start and completion events. *Done when:* a scheduled scan runs, reports progress, and stays inside the throttle.

6.2 The backend — Alex

A1.1 — New backend module `backend/modules/edr/`. Follow the existing module shape exactly (`connectors`, `domain`, `dto`, `handler`, `repository`, `usecase`, `module.go`, `routes.go` — copy the structure of `backend/modules/datasources/`).

Endpoints, all under `/api/v1/edr`:

Method and path	Purpose
GET /overview	Fleet summary: endpoints with the module, healthy, unhealthy, stale signatures, detections in the last day
GET /endpoints	Per-endpoint list with status, policy, last report time
GET /endpoints/:agentId	Full stored status for one endpoint
POST /endpoints/:agentId/actions	Dispatch one action (scan, kill, isolate, release, refresh)
GET /policies, POST, GET /:id, PUT /:id, DELETE /:id	Policy documents
PUT /policies/:id/assignments	Assign a policy to endpoints or groups
GET /quarantine	Quarantine items across the fleet, filterable by endpoint
POST /quarantine/:itemId/restore, POST /quarantine/:itemId/purge	Act on one item
GET /blocklist/status	Indicator counts, feed freshness, recent blocks
GET /blocklist/allowlist, POST, DELETE	Network allowlist entries

A1.2 — Permissions and migration. Add `edr.read` and `edr.write` to the permissions table following the pattern in `backend/migrations/000001_init.up.sql` (a new numbered migration, with the matching removal in the down migration), and gate every route with `middleware.RequirePermission`. *Done when:* a user without the permission gets a refusal on every route, and a fresh install has the permissions seeded.

A1.3 — Extend the agent-manager client. `backend/pkg/agentmanager/client.go` already wraps the panel service. Add the calls the new messages need, with sensible timeouts and a clear error when the agent is offline. *Done when:* every backend route that touches an endpoint returns a useful error within ten seconds when the endpoint is unreachable.

A1.4 — Storage. Backend owns policies and assignments. Agent-manager owns the latest status per agent and the command log (it already has an `AgentCommand` table — extend rather than duplicate). Define which side is the source of truth for each table and write it down in the module's README. *Done when:* a server restart loses no policy and no status older than one reporting interval.

A1.5 — Assistant tools. Add EDR entries to `backend/modules/mcp/catalog.json` and the matching tool file (`backend/modules/mcp/tools_edr.go`, following `tools_datasources.go`): read fleet status, read one endpoint, list quarantine, list recent detections. Read-only first; any action tool needs Rick's explicit approval before it ships. *Done when:* the assistant can answer "which endpoints are unprotected right now" from real data.

6.3 The console — Andres

New feature folder `frontend/src/features/edr/`, registered in `frontend/src/app/routes/index.tsx` and in the navigation, gated on `edr.read` and `edr.write`, fully translated.

Three requirements shape every screen here, and none of them is optional.

Every endpoint setting is managed from the console. The complete surface — sensors, allowlists, fail mode, freeze-on-launch and its timeout, quarantine retention, scan concurrency, watched volumes, signature source and fallback, engine tier, the whole ransomware block, the whole network-blocking block, the scheduled scan, isolation exceptions, and the Linux permission mode. The authoritative list is `agent/edr/config/config.go` and it is reproduced in full in issue N1.3. An administrator should never need to touch a file on an endpoint.

Configuration is differential. A policy targets all endpoints, an asset group, or a named selection. The more specific layer wins — endpoint beats group beats fleet-wide — and the console shows the resolved outcome per endpoint before anything is saved. Layering nobody can see the result of is worse than no layering.

A dedicated dashboard, not a widget. Fleet posture at the top, activity in the middle, and a per-workstation health grid at the bottom that an administrator can scan to find the three bad machines out of a hundred. This is the screen a customer will compare against a purpose-built endpoint product's console.

N1.1 — Fleet overview page (`pages/EdrOverviewPage.tsx`). Coverage (how many endpoints have the module, how many are missing it), health, signature freshness, detections over time, ransomware and network-block counters, and the list of endpoints needing attention.

N1.2 — Endpoint detail panel (`components/EdrEndpointDrawer.tsx`), reachable both from the overview and from the existing data sources page where agents already live (`frontend/src/features/datasources/pages/DataSourcesPage.tsx`). Shows status, which sensors are on, engine tuning, applied policy and drift, recent detections, quarantined files, and the action buttons: scan, full scan, kill process, isolate, release, refresh indicators.

N1.3 — Policy editor (`pages/EdrPoliciesPage.tsx` plus `components/PolicyEditor.tsx`). Create, edit, assign. Grouped sections: sensors, allowlists (paths, processes, commands, networks), ransomware response mode and thresholds, network blocking, quarantine retention, scheduled scan, signature source. Every dangerous setting needs an explanation in plain words — particularly the freeze-on-launch switch, which is off for a reason, and the ransomware kill mode.

N1.4 — Quarantine browser (`pages/EdrQuarantinePage.tsx`). Fleet-wide list: endpoint, path, detection name, hash, quarantine time, restore and permanent-delete actions with a confirmation and an audit note.

N1.5 — Network blocking page (`pages/EdrBlocklistPage.tsx`). Indicator counts by type, feed freshness, recent blocked connections, and the allowlist editor.

N1.6 — Isolation is visible and obvious. An isolated endpoint must be unmistakable in every list it appears in, with a one-click release and a clear note about what isolation still allows.

Done for all of the above when: Jose can drive a complete incident — see the detection, inspect the endpoint, isolate it, restore a file, release it — without leaving the console and without reading a log file.

6.4 Company-wide false-positive management — Alex and Andres

Every company runs software the scanner will flag that is not malicious: an in-house build tool, a licence checker, a bespoke script. Today the only fix is editing a file on each affected machine. That does not scale, and it means the first false positive on a busy day gets "solved" by someone switching the module off.

What gets built (issues A1.7 for the backend, N1.7 for the screens):

- A **company-wide allowlist** the whole fleet inherits, layered like policies are: company-wide, group, or one endpoint. Entry kinds match what the endpoint already understands — file path, program image, command text, network address — plus file hash, because allowlisting one exact build is safer than allowlisting a path anyone can write to.
- **Promote a detection to an allowlist entry in one action.** From any EDR detection, the administrator confirms how widely it applies and why, and it is done.
- **Restore the file everywhere it was quarantined**, in the same action. A false positive usually hits many machines at once, and cleaning up one at a time is the part people give up on.
- **An audit trail on every entry** — who, when, why, and which detection it came from — with an optional expiry, so a temporary exemption does not become permanent by accident.
- **Limits on over-broad entries.** An allowlist covering a drive root or the system directory is refused with an explanation, not silently narrowed.

The endpoint rule still holds: these entries are **additive** to the built-in lists that ship with the product and never replace them.

6.5 Verification — Jose

J1.1 — Windows acceptance of the control path. Every action, every failure case, on a real Windows machine: module absent, module present but stopped, endpoint offline mid-command, policy conflict between local and central.

J1.2 — Coexistence with Windows Defender. Two real-time scanners on one machine fight. Run the full scenario set with Defender on, and record what behaves differently and what we must tell customers. This has never been done properly — past testing turned Defender off for clean attribution.

J1.3 — Response playbooks. Write the automated responses in `definitions/soar/`: isolate on contained ransomware, alert and collect on repeated blocked connections, notify on a sensor turned off. Test each one end to end.

J1.4 — False-positive pass. Run the module for a week on ordinary working machines — developer, accounting, backup server — and tune the built-in allowlists from what actually fires.

7. Phase 2 — Linux parity (8–10 weeks)

Everything the Windows module does, done again with Linux mechanisms. The module already compiles on Linux with every sensor stubbed out, so the shape exists; the sensors do not.

One genuine upgrade to call out: on Linux, `fanotify` permission events let us decide whether a file may be opened or executed **before** it runs. On Windows the module can only detect and then kill, with a few milliseconds of exposure. The Linux build can therefore be strictly stronger, and the product wording should say so.

Yadian — the sensor core

Y2.1 — Service and layout. systemd unit, install and uninstall verbs, install directory, file ownership and permissions, log location. Mirror the Windows service lifecycle so the agent controls it identically.

Y2.2 — File watch through fanotify. Replace the change-journal watcher. Start in notify mode (detect and quarantine, matching Windows), then add permission mode as a policy option so a malicious file can be refused at open or exec time. Handle mount points, bind mounts, network file systems and container overlays deliberately — this is where the sharp edges live.

Y2.3 — Process watch through the netlink process connector. Replace the Windows management interface feed. Must deliver process start with identifier, parent identifier, image path and command line, and must not miss short-lived processes the way the current Windows poll can.

Y2.4 — Kill and freeze. Terminate a process tree with signals, leaves first, reusing the existing tree logic. Freeze with the control-group freezer rather than per-process stops.

Y2.5 — Ransomware guard on Linux. Reuse the scorer and the decoy logic unchanged; replace the file-activity feed with fanotify and replace the Windows recovery-tampering rules with the Linux equivalents: deleting or shredding backup trees, stopping backup services, removing file system snapshots, disabling journaling.

Y2.6 — Paths, mounts and volume enumeration. The Windows code assumes drive letters in several places. Audit every path comparison, the allowlist matcher, quarantine storage and the decoy placement for correct behaviour with symbolic links, hard links and case-sensitive file systems.

Alex — the pieces that are not sensor work

A2.1 — Network blocking on Linux through nftables: its own table and chain so we never disturb the customer's firewall, the same self-lockout protection, the same fail-open behaviour when the module stops, and the same event on a drop.

A2.2 — Engine packaging for Linux. Per-distribution-family and per-architecture packages, hosted and downloaded the same way as the Windows package from A0.2.

A2.3 — Build pipeline for Linux binaries, published at the dependencies endpoint for both Intel/AMD 64-bit and ARM 64-bit.

A2.4 — Backend and interface work for anything Linux-specific that the screens need: a different sensor set, the permission-mode option, distribution and kernel version in the endpoint detail.

Andres — console

N2.1 — Make every screen operating-system aware. Sensors that do not exist on Linux must not appear as "off"; they must not appear at all. The policy editor must show the Linux-only permission-mode option and hide the Windows-only script-scanning switch.

N2.2 — Mixed-fleet views. Coverage and health must read correctly when a customer runs both operating systems.

Jose — verification

J2.1 — Capability matrix. Build a table of every capability against Windows and Linux with a verified result in each cell, on real machines, not in a container. Cover at least two distribution families and both processor architectures.

J2.2 — Linux detection content. Extend the parser and rules for the Linux-only fields and behaviours, and add rules for the Linux recovery-tampering signals.

J2.3 — Performance under load. Measure processor and memory cost of the file watch on a busy build server and a busy database server. Ransomware decoys and fanotify are both capable of hurting a loaded machine; find the limits and set safe defaults.

8. Phase 3 — Production hardening (4 weeks, overlapping)

Task	Owner	Detail
H1 — Authenticode signing	Alex	Buy or locate the certificate, sign <code>utmstack_edr.exe</code> and <code>utmstack_amsi.dll</code> for both Windows architectures as a pipeline step. The script-scanning library in particular is loaded into other processes and will be distrusted unsigned
H2 — Script blocking on Intel/AMD 64-bit	Jose	The native library has only ever run on an ARM machine. Prove it on Intel/AMD hardware
H3 — Known ransomware guard defects	Yadian	The culprit's image is sometimes empty when the process table lags; a containment event can be emitted twice; a process that finishes in under a second can slip past the process poll
H4 — Known network blocking defects	Alex	The drop event reports process identifier zero and always reports the direction as outbound; device paths are not always converted to readable paths
H5 — Configuration safety	Yadian	Hand-editing <code>edr.json</code> to switch network blocking off is silently ignored today — a trap for a support engineer. Also expose the signature fallback setting in the command-line tool
H6 — Scale test	Jose	One hundred endpoints reporting status and pulling signatures from one server. Measure the mirror's load and the event volume the behavioural sensor produces
H7 — Upgrade and rollback	Yadian	Prove an endpoint upgrades from one module version to the next without losing quarantine, configuration or the event spool, and that a failed upgrade leaves a working agent
H8 — Customer-facing documentation	Andres with Jose	What the module does, what it cannot do, how to tune a false positive, what isolation allows through. The honesty rule applies: we detect and kill quickly on Windows, we can genuinely prevent on Linux, and script scanning only sees what the script host chooses to submit
H9 — Effectiveness campaign	All four	See below

The effectiveness campaign (H9) — everyone takes part

Once everything is built, we find out whether it actually works. **All four engineers take part**, because four people looking at it from four angles find what one person misses. Yadian and Jose lead, since they know the agent's inner workings best.

Engineer	Their part
Jose	Coordinator and detection effectiveness. Designs the test set, runs real malware, live ransomware in an isolated network, malicious scripts, drop-and-run payloads and known-bad connections against both operating systems. Owns the final report
Yadian	Endpoint depth and evasion. Attacks the module itself — stop the service, delete the quarantine store, tamper with the configuration, flood the event spool, kill the engine, start a process faster than the watcher sees it, encrypt faster than the scorer escalates. Every gap is fixed or written down as a known limit
Alex	Server and delivery under pressure. Signature updates failing mid-attack, a hundred endpoints reporting at once, a stale mirror, backend throughput when every endpoint reports together
Andres	The console under real conditions. Drives a full incident using only the console. Anything needing a shell, a log file or a database query to understand is a defect

The rule for the report: **measure outcomes, not logs**. A log line saying a process was killed is not evidence it was killed — look at the process list, the quarantine store, the file system and the alert. The output is one honest document covering what we catch, what we do not, how fast, and what a customer must not be told we do. It feeds the documentation and it is the basis for how this gets sold.

8b. Phase 4 — Next capabilities (by Fri 15 Jan 2027)

Work deferred on purpose, but not left open-ended. Each of these has an owner and a date.

Task	Owner	Due	Detail
X1 — Ransomware sensors, second round	Yadian	Fri 18 Dec	The guard ships with two signals: decoy files and recovery-tampering commands. Real families reliably trip neither. Adds content randomness, extension churn, modification rate, ransom-note detection and registry tampering, feeding the existing scorer. No single signal may escalate alone
X3 — Deferred defect backlog	Alex	Fri 11 Dec	The small known defects recorded and deliberately postponed during development, on both the server and endpoint modules. Each fix needs a test that fails first
X2 — Application control	Alex	Fri 15 Jan	The one capability the original requirement document scoped that this programme left out, and the only route to true prevention on Windows . Design, prototype and a recommendation — not a commitment to ship. Built on the operating system's own feature; still no kernel driver. The hard part is not enforcement, it is giving a customer a workable policy without a month of effort. Concluding "too risky to roll out" is a successful outcome

9. Rules everyone follows

1. **Branding.** Every event field and every surfaced log line says "UTMStack EDR". The scanning engine's vendor name must never appear in an emitted field or a log message. There is a grep-based audit; keep it green.
 2. **Quarantine, never delete.** Files move to a protected store with a restore path. Permanent deletion is only ever an explicit administrator action.
 3. **No kernel driver.** Windows response uses ordinary process termination with system privileges; Linux uses signals and fanotify. Pre-execution blocking on Windows would need the built-in application control feature, which is a later phase.
 4. **Fail open by default.** If the module dies, the endpoint keeps working: filters release, isolation lifts, files open.
 5. **Honest capability claims.** Do not describe the Windows file and process path as prevention. It is fast detection and termination with a small window of exposure.
 6. **The endpoint works offline.** Last-known policy, last-known signatures, last-known indicators. Stale, never empty.
 7. **Test on real machines.** Every Windows-specific and Linux-specific path has produced real bugs that only appeared on a real machine — twelve of them so far. Check the final result (the quarantine store, the event at the platform), never just the log line.
-

10. Decisions needed from Rick before Phase 1 starts

1. **Control path shape.** Typed messages on the agent stream, as recommended above, or the faster route of sending command-line strings through the existing shell channel? The typed route costs about a week more and gives a real audit trail and real error handling.
 2. ~~When do the two branches go to the shared repository?~~ **Settled 21 Sep:** `utmstack/OpenEDR` is the working home; both branches are pushed and merged into `main`, which is the default branch.
 3. **Linux scope.** Which distribution families and which minimum kernel version? Permission-mode blocking needs a reasonably recent kernel.
 4. **macOS.** In or out? The agent supports it; the EDR module has no macOS work at all and it is not in this plan.
 5. **Code-signing certificate.** Who procures it, and does it cover the native library as well as the module?
 6. **Licensing.** Is the EDR metered or licensed separately from data sources? The billing module exists and would need to know.
-

11. Risks

Risk	Impact	What we do about it
Two real-time scanners on one Windows machine	Unpredictable attribution, customer confusion	J1.2 coexistence pass, and clear guidance in the documentation
fanotify performance on busy Linux servers	The module becomes the reason a server is slow	J2.3 measured limits before defaults are set
Network blocking is on and enforcing by default	A bad indicator list could cut a customer off	Self-lockout protection already exists; add an emergency central switch and make it findable
Signature mirror not deployed at a customer	Endpoints fall back to the public network or go stale	Fallback already built; make staleness visible in the console
The endpoint work sits unmerged	Drift against <code>release/v12.0.0</code> grows, and one machine holds the only copy	Phase 0 merges first; a private backup already exists
Behavioural telemetry volume	It can crowd out security events and inflate storage	Sensor switch already exists; measure in H6 and set a sane default

12. The dated task list

Every task below is a GitHub issue in `utmstack/OpenEDR`, assigned to its owner, carrying its milestone and its due date, and tracked on the org board at <https://github.com/orgs/utmstack/projects/2>.

Dates are commitments, not estimates. They are deliberately tight: the team is using AI assistance heavily, which removes most of the typing and a good deal of the reading, so the schedule assumes that productivity rather than a traditional one.

The order has been checked: no task is due before something it depends on, and no task shares a due date with a dependency it has to consume first.

Workload: Yadian 18 tasks, Alex 18, Jose 12, Andres 11, plus the effectiveness campaign all four share.

Phase 0 — Land what exists (all done by Fri 2 Oct)

Due	Task	Owner	What
Wed 23 Sep	Y0.1	Yadian	Review and merge the endpoint module branch into edr-develop
Thu 24 Sep	A0.1	Alex	Publish the endpoint binaries through the build pipeline
Thu 24 Sep	J0.1	Jose	Write the platform parser for EDR events
Fri 25 Sep	Y0.2	Yadian	Review and merge the master-server EDR container branch
Tue 29 Sep	J0.2	Jose	Write the first twelve EDR correlation rules
Wed 30 Sep	A0.3	Alex	Deploy the EDR server container on a real deployment and prove both transports
Thu 1 Oct	J0.3	Jose	Set alert severity and scoring for EDR detections
Fri 2 Oct	A0.2	Alex	Package the scanning engine and make the endpoint install it by itself
Fri 2 Oct	N0.1	Andres	Present EDR detections properly in alerts and the log explorer

Phase 1 — Console control on Windows (all done by Fri 23 Oct)

Due	Task	Owner	What
Mon 5 Oct	A1.2	Alex	Add EDR permissions and the database migration
Tue 6 Oct	Y1.1	Yadian	Define the typed control message contract between the server and the agent
Thu 8 Oct	A1.1	Alex	Build the backend EDR module and its interface
Thu 8 Oct	Y1.2	Yadian	Add machine-readable output to the endpoint module's command-line tool
Fri 9 Oct	J1.2	Jose	Run the Windows Defender coexistence pass
Fri 9 Oct	N1.1	Andres	Build the dedicated EDR dashboard with per-workstation health
Tue 13 Oct	A1.3	Alex	Extend the agent-manager client for the EDR messages
Tue 13 Oct	Y1.3	Yadian	Build the agent-side handler for EDR commands
Wed 14 Oct	N1.2	Andres	Build the endpoint detail panel with its actions
Thu 15 Oct	A1.4	Alex	Settle and implement the EDR storage model
Thu 15 Oct	Y1.4	Yadian	Report endpoint status upstream
Fri 16 Oct	A1.6	Alex	Build host isolation
Fri 16 Oct	N1.3	Andres	Build the configuration editor — every setting, applied to any set of endpoints
Tue 20 Oct	A1.5	Alex	Add EDR tools to the assistant catalogue
Tue 20 Oct	N1.4	Andres	Build the fleet quarantine browser
Tue 20 Oct	Y1.5	Yadian	Build central policy distribution
Wed 21 Oct	A1.7	Alex	Build company-wide allowlist and false-positive management (backend)
Thu 22 Oct	J1.1	Jose	Accept the console control path on Windows
Thu 22 Oct	N1.5	Andres	Build the network blocking page
Thu 22 Oct	N1.7	Andres	Build the allowlist and false-positive screens
Fri 23 Oct	J1.3	Jose	Write the automated response playbooks
Fri 23 Oct	J1.4	Jose	Run the false-positive tuning week
Fri 23 Oct	N1.6	Andres	Make isolated endpoints unmistakable everywhere
Fri 23 Oct	Y1.6	Yadian	Add a scheduled full-disk scan

Phase 2 — Linux parity (all done by Fri 20 Nov)

Due	Task	Owner	What
Mon 26 Oct	A2.3	Alex	Build and publish the Linux endpoint binaries
Wed 28 Oct	Y2.1	Yadian	Bring up the endpoint module as a Linux service
Fri 30 Oct	A2.2	Alex	Package and host the scanning engine for Linux
Fri 6 Nov	A2.1	Alex	Build Linux network blocking on nftables
Fri 6 Nov	Y2.2	Yadian	Build the Linux file watcher on fanotify
Wed 11 Nov	A2.4	Alex	Extend the backend and its interface for Linux endpoints
Wed 11 Nov	Y2.3	Yadian	Build the Linux process watcher on the netlink process connector
Fri 13 Nov	N2.1	Andres	Make every EDR screen operating-system aware
Fri 13 Nov	Y2.4	Yadian	Implement process termination and freezing on Linux
Tue 17 Nov	Y2.6	Yadian	Audit every path assumption for Linux
Wed 18 Nov	J2.3	Jose	Measure the cost of the Linux sensors under real load
Wed 18 Nov	N2.2	Andres	Make the fleet views correct for mixed environments
Wed 18 Nov	Y2.5	Yadian	Port the ransomware guard to Linux
Fri 20 Nov	J2.1	Jose	Complete the cross-platform capability matrix
Fri 20 Nov	J2.2	Jose	Extend the parser and rules for Linux

Phase 3 — Production hardening (all done by Fri 4 Dec)

Due	Task	Owner	What
Wed 25 Nov	H2	Jose	Prove script blocking on Intel and AMD hardware
Wed 25 Nov	H3	Yadian	Fix the three known ransomware guard defects
Fri 27 Nov	H1	Alex	Sign the Windows binaries
Mon 30 Nov	H4	Alex	Fix the three known network blocking defects
Mon 30 Nov	H5	Yadian	Close the configuration safety trap
Wed 2 Dec	H9	Alex + Jose + Yadian + Andres	EDR effectiveness campaign — all four engineers
Thu 3 Dec	H6	Jose	Run the scale test
Thu 3 Dec	H7	Yadian	Prove upgrade and rollback
Fri 4 Dec	H8	Andres	Write the customer-facing documentation

Phase 4 — Next capabilities (all done by Fri 15 Jan 2027)

Due	Task	Owner	What
Fri 11 Dec	X3	Alex	Close the deferred defect backlog
Fri 18 Dec	X1	Yadian	Ransomware guard — the second round of sensors
Fri 15 Jan	X2	Alex	Application control — design, prototype and decide

13. Nothing is left without an owner

Every item recorded as pending anywhere in this project's documents now has a named engineer and a date:

Pending item, and where it was recorded	Now owned by
Signature mirror on the customer's server (gap analysis §1.1)	Alex — A0.3
Threat-intelligence mirror on the customer's server (gap analysis §1.2)	Alex — A0.3
Hosting the endpoint binaries at the dependencies endpoint (gap 3)	Alex — A0.1, A2.3
Engine provisioning on the endpoint (gap 4)	Alex — A0.2, A2.2
Signature-mirror client polish (gap 5)	Shipped; the last piece, the fallback setting in the command-line tool, is in Yadian's Y1.2
Network feed client polish (gap 6)	Shipped; upstream path confirmation is in Alex's A0.3
Authenticode signing (gap 7)	Alex — H1
Script blocking on Intel/AMD hardware (gap 8)	Jose — H2
Platform parser for EDR events (gap 9)	Jose — J0.1
Correlation rules and response playbooks (gap 10)	Jose — J0.2, J0.3, J1.3, J2.2
Ransomware guard known defects (gap 11)	Yadian — H3
Network blocking known defects (network blocklist notes)	Alex — H4
Configuration safety trap, hand-edited settings ignored (network blocklist notes)	Yadian — H5
Deferred minor defects on both modules (progress ledgers)	Alex — X3
Second round of ransomware sensors (ransomware design document)	Yadian — X1
Application control (requirement document; excluded from Phase 1 by design)	Alex — X2
Scheduled full-disk scan (found missing in this review)	Yadian — Y1.6
Host isolation (found missing in this review)	Alex — A1.6
Company-wide false-positive management (Rick, 21 Sep)	Alex — A1.7, Andres — N1.7
Full configuration surface in the console (Rick, 21 Sep)	Andres — N1.3
Dedicated per-workstation dashboard (Rick, 21 Sep)	Andres — N1.1

Pending item, and where it was recorded	Now owned by
Effectiveness testing with everyone taking part (Rick, 21 Sep)	All four — H9
Linux support across every capability	Yadian (sensors), Alex (network, packaging), Andres (screens), Jose (verification) — the Phase 2 tasks

The only item deliberately left without a task is **macOS**. The agent supports it; the EDR module has no macOS work and none is committed. It is decision 4 in section 10.